The book of seahtrue

Vincent de Boer

2025-06-27

Table of contents

Preface

This is the R Seahorse data analysis manual using functions from the Seahtrue package. Its purpose is to demonstrate and educate how to use R for Extracellular Flux analysis. We use a dedicated data analysis pipeline for quality control and advanced plotting of the data.

You can run R this manual in your browser and do not need to install any R or Rstudio software on your computer.

> **Note**
>
> The manual is targeted to all levels of learning, meaning that also interested learners without any R background or programming knowledge can use it

> **Tip**
>
> To turn your `.asyr` Seahorse Wave file into an excel `.xlsx` file, you can use the Seahorse Wave desktop software or the Seahorse analytics website from Agilent:
> https://seahorseanalytics.agilent.com
> Lots of info regarding Seahorse analysis including how it works and how to run experiments is available from the Agilent website
> All things lab - How to run an assay:
> https://www.agilent.com/en/product/cell-analysis/how-to-run-an-assay
> All other Agilent Seahorse info:
> https://www.agilent.com/en/products/cell-analysis/how-seahorse-xf-analyzers-work

Resources

Free books

Telling Stories with data

Excellent very complete overview covering basic R coding, communicating science, and statistics.
These two chapters are particularly good:

- Clean and prepare https://tellingstorieswithdata.com/09-clean_and_prepare.html

- appendix 1: R essentials https://tellingstorieswithdata.com/20-r_essentials.html

R for data Science

The go-to book when you want to get familiar with R and tidy from Hadley Wickham and others from Posit https://r4ds.hadley.nz/

Nice sections, for example:

- ggplot basics https://r4ds.hadley.nz/data-visualize
- lubridate date and time basics https://r4ds.hadley.nz/datetimes

Functional programming book

Nice visualizations of data structures and operations

- vectors, lists, tibbles https://dcl-prog.stanford.edu/data-structure-basics.html
- map and purrr https://dcl-prog.stanford.edu/purrr-basics.html

Fundamentals of Data Visualization

Everything you want to know about visualizing data using ggplot, with beautifull plots. Also, all code fully available on github

- associations (scatter plots etc) https://clauswilke.com/dataviz/visualizing-associations.html
- uncertainty (error bars and distirbutions etc) https://clauswilke.com/dataviz/visualizing-uncertainty.html
- ggplot tutorial by Cedric Scherer https://www.cedricscherer.com/2019/08/05/a-ggplot2-tutorial-for-beautiful-plotting-in-r/

Webr/WASM

webr REPL - a full complete rstudio-like R environment in the browser https://webr.r-wasm.org/latest/

Webr/wasm presentation bob rudis - (how I got interested in webr/wasm) - https://www.youtube.com/watch?v=inpwcTUmBDY

Webr manual https://docs.r-wasm.org/webr/latest/

Webr on github https://github.com/r-wasm/webr/

Other info

Learn to purrr, Rebecca Barter - https://www.rebeccabarter.com/blog/2019-08-19_purrr

Movies:

Purrr and map functions explained by Hadley - https://www.youtube.com/watch?v=EGAs7zuRutY

R fun

https://twitter.com/rafamoral/status/1571622591219236864?s=20&t=RJWOSe30-8bbDxgLIamRUQ

Part I

R concepts - Jump into the water

In this section we learn how to swim in R code. We get ourselves familiar with using R for data handling and plotting. We will use first a dataset from R itself and secondly work with a Seahorse data file. There are exercises, with solutions, as well.

1 Ditching point-and-click and diving into R

Data analysis in biological and medical sciences is dominated by the use of point-and-click tools (like Excel, Prism, SPSS, etc.). The reasons for this are that it is easy, it is visual, and gets you to a fast outcome. These point-and-click tools are convenient to use and their main asset is that you have a canvas or grid in front of you for dragging-dropping, copy-pasting, typing and calculating. For plotting data, you can choose formatting options by clicking on the features you want to change, again on a canvas that is in front of you on the screen. T 1his canvas style of working is likely what is closest to our natural way of getting things done; putting stuff together with your hands and seeing directly what happens to the stuff is quick and actionable.

The disadvantages of using point-and-click tools are that they are not traceable and can be prone to mistakes. The sequence of clicks and manipulations the data went through doesn't get recorded, which makes the process not reproducible for others and, most importantly, for you. Did you ever felt frustrated for having to manually generate and copy-paste a collection of figures? Then you are already familiar with these disadvantages. Also, you will always work within the limits of the tools that you are using, or how Bruno Rodrigues, the author of the free book Building reproducible analytical pipelines with R, phrased it "… point and click never allow you to go beyond what vendors think you need." X-link.

Enter R! R is possibly the best, easiest and most accessible tool to use for biologists and like-minded scientists.
R is part of or adopted more and more in scholarly programs at academic institutes not only for statistical use but also for other aspects of data science. Other tools like Python and Matlab are also widely used and they have similar benefits as R over point-and-click tools. Matlab however is proprietary software that needs high license fees from institutions to be able to work with it.

Here you will jump straight to using the `tidyverse` way of working https://www.tidyverse.org/. A complete (but extensive) overview of R data science can be found at https://r4ds.hadley.nz/. The "R for data science" resource also centers around the `tidyverse` and the `tidy` concept of data handling. Often data handling (organizing the data and tidying it to be able to use it in your downstream workflow) is describes as `data wrangling`. As mentioned in the `R for data science book`: "Together, tidying and transforming are called wrangling because getting your data in a form that's natural to work with often feels like a fight!" https://r4ds.hadley.nz/intro. With the `tidyverse` and some level of experience your fight will become easier over time.

Since the R community is huge, there is also an overwhelming number of resources (like books, tutorials, videos, blog posts, stack-exchange content, …) that all want to teach, educate and inform you about R in one way or the other. Also, there are again collections of R resources, and even collections of collections of R resources, …

These tutorials and courses have one thing in common, they start of with installing R and Rstudio and learning the software. How nice would it be if we can skip these (often) nasty installations? What if we can skip version updating and package installations and start working with your data right away? That would be amazing! And this is possible with the development of `webr` by George Stagg and colleagues https://github.com/r-wasm/webr.

> It is R in the browser!

This is so great, because it provides the most convenient, quick and easy way to enter the R world. It is just like you having your Excel, Word and Powerpoint always immediately up and running by a click of a button. Since with R we type in our commands instead of pointing and clicking we are in the era of type-and-click to get your data science done.

This book is completely written using `quarto` and `webr` and allows you to typ in the code and run it right in the browser.

2 Jumping essentials

2.1 The essentials

Before you jump into the water it can be of benefit when you know more about the water. What is the temperature? Is it really cold or just nice and warm? How high is the jump? Do you need to jump first 5 meters from a diving board or can you already feel the water with your toes?

This first chapter will give some basic programming essentials that will allow you to jump easier. Also it can be used as a reference for when you need to make the jump again

2.1.1 Find your info online and in documentation

R has so many functions that it is impossible to know everything by heart. Documentation and the internet are always your best friend.

`Stackexhange` is an excellent resource. 90 to 99% of your questions related to how you should use your R and tidy functions has been asked before by others. The nice thing is that, most often, the questions include a snippet of code that makes the problem reproducible. More importantly, almost all questions have been accurately answered in multiple ways.

Other resources that often come up in my search results are either forums on `POSIT community`, `Reddit`, or `Github discussions or issues`. These are more forum-like comments, with not such a good solvability structure as stackexchange.

Then there are many more resources that somehow scrape the internet and collect basic info. Most of the time the info is correct but too simplistic. Not real issues are tackled. These are sites like `geeksforgeeks`, `datanovia`, `towardsdatascience`, some have better info then others, but most of the time these have commercial activities and in the end want to sell you courses or get your clicks.

The good news is that **you don't have to remember any of these sites by heart. Just type your question on your favorite search engine**, ideally copypasting the exact error message, and your answer would very likely be found.

2.1.2 R and tidyverse documentation

All functions in R and tidyverse are accurately documented. All its arguments are described and especially the `examples` that are given are really helpful. Packages have often even more documentation called `vignettes` that explain certatin topics and contexts on how and when to use the functions.

2.1.3 Style and layout

Writing your code benefits from proper readability. Just like we layout our texts, manuscripts and excel data files, we also need a good layout for our code.

```
library(ggplot2)

# NOT VERY READABLE (but runnable )
  ggplot(data=mtcars,   mapping=aes   (x = mpg,y   = disp,
color = hp,shape =             as.factor(cyl))         ) +geom_point()
```

There are mulitple ways to organize your code, I try to adhere to: - short lines (max 60 characters per line) - indent after first line - indent after ggplot - each next function call aligns with the above function - each argument aligns with the previous argument - each ggplot layer gets its own line - I put the x and y aesthetics for ggplot mapping on one line

Other good practices are: - use the package name before a function, like `dplyr::mutate` - use comments to annotate the code, when you put a `#` before it, it is not executed

So here is an example on what not to do and its corrections

```
library(dplyr)
#NOT GOOD
iris %>%
as_tibble() %>% janitor::clean_names() %>%
filter(species
       %in% c("setosa", "virginica")) %>%
        ggplot(aes(x = sepal_length, y = sepal_width,group = petal_length, color = petal_wid
geom_point()+ geom_line() +
    theme_bw(base_size = 16)

#GOOD
iris %>%
  as_tibble() %>%
  janitor::clean_names() %>%
```

```
  filter(species %in% c("setosa", "virginica")) %>%
  ggplot(aes(x = sepal_length, y = sepal_width,
             group = petal_length,
             color = petal_width))+
    geom_point()+
    geom_line() +
    theme_bw(base_size = 16)
```

2.1.4 Calling packages and functions

There are two ways of calling functions in R, the most straightforward and easy one is just to call it with its name. This works only when you run a function from base R. When you want a function from another package, you can either first load the package with `library(your_favorite_package)` and then call your function with `my_favorite_function(my_argument)`. Another, preferred way, is to always explicitly mention the package from which the function comes from. It can happen that two different packages implement a function with the same name, leading to confusion. In that case, you need to be careful with the `library` loading, because then the function might be masked by another function with the same name from another package

```
#option 1
library(janitor)
clean_names(iris) %>%
  head()

#option 2 (preferred)
janitor::clean_names(iris) %>%
  head()

#which is the same as:
iris %>%
  janitor::clean_names() %>%
  head()
```

2.2 Basic R semantics

When starting using R and tidyverse the new language can be daunting. So here is a short primer of common semantics that are often not directly understood from code.

I took some of these example directly or indirectly from:

https://uc-r.github.io/basics

2.2.1 Assignment

The most common way of assigning in R is the `<-` symbol. Although the `=` works in the same way, it is reserved by R users for other things. I tend to use it for assigning numbers to constants, and it is used in function arguments

```
#assignment
x <- 1

#is the same as:
x = 1

#but the <- is preffered

print(paste0("my x = :", 1))
```

2.2.2 Vectors and lists

A `vector` in R is a collection of items (elements) of the same kind (types). A `list` is a collection of items that can also have different types. We make a vector with `c()` and a list with `list`. The `c` in `c()` apparently stands for `combine` link

```
#vectors
x <-  c(1, 2, 3)
y <-  c ("aap", "noot", "mies")
x
y
```

```
#lists
x <-  list(1, 2, 3)
y <-  list("aap", "noot", "mies", 1, c(22, 23, 25))
x
y
```

If you try to build a vector with elements of different types, R will try to adapt all of them to a single type. You can see that when you specify a vector with numbers and characters eg. `c(1, 2, "1", "2)`. It forces the vector to be of `character` type. While it may look handy that R does this for us, it is a dangerous feature that might lead to wrong inputs going unnoticed.

```
#other vector semantics
x <- 1:10

#is the same as
x <- c(1:10)

#is the same as
x <- c(1,2,3,4,5,6,7,8,9,10)

#you can multiply all elements of a vector at the same time
x * 3

# or:
y <-  3
x * y

# or:
x / y

# also adding y to x will add 3 to each element
x + y


# you can also extend or combine two vectors
z <- 20:25

c(x, z)
```

Lists form the basis of all other data than vectors. Dataframes are collections of related data with rows and columns and unique columns names and row names (or row numbers). `data.frame` is actually a wrapper around the `list` method.`Tibbles` are the tidyverse equivalent of `dataframes` with some more handy properties over dataframes. A 'list' can have names items or not.

```
#a list without named items
my_list <- list(1:10, letters[1:10], LETTERS[1:10])

#a list with named items
my_list <- list(my_numbers = 1:10,
                my_lowercase = letters[1:10],
                my_uppercase = LETTERS[1:10])

#this almost looks like a table, it only is not in a matrix format

#turning the list into a dataframe generates a table
as.data.frame(my_list)

#which is similar to making it a tibble
as_tibble(my_list)

#when the columns are not of the same length the df or tibble
#cannot be generated
my_list_2 <- list(my_numbers = 1:10,
                my_lowercase = letters[1:10],
                my_uppercase = LETTERS[1:10])

as.data.frame(my_list_2)
```

2.2.3 Common semantics

R language is different from other programming languages, and when starting out learning R there are some rules and common practices.

2.2.4 ~ (the "tilde")

```
#the primary use case is to separate the left hand side
#with the right hand side in a formula

y ~ a*x+ b

#the ~ is also used in the ggplot facet_wrap or facet_grid
```

```
#it can be read as "by"
# separate the ggplot "by" cyl
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point()+
        facet_wrap(~cyl)

#the tilde use in the facet_wrap is discouraged though,
#we now use vars() instead of the tilde

mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point()+
        facet_wrap(vars(cyl))
```

2.2.5 + (the plus)

Apart from the simple arithmetic addition, + is also used in the ggplot functions. It adds the multiple layers to each ggplot

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point()+
        geom_line()+
        geom_boxplot()+
        labs(title = "Crazy plot")
```

2.2.6 %>% (the pipe)

The `%>%` is used to forward an object to another function or expression. It was first introduced in the `magrittr` package and is now also introduced in base R as the `|>` pipe, which are now identical. See blogpost for more info.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    mutate(new_column = mpg*cyl) %>%
    filter(new_column > 130)
```

2.2.7 == (equal to)

The `==` is the equal to operator. It is different than `=` which is used only for assignment.

```
#the equal to is validating whether the left hand side
#is the same as the right hand side and its output is TRUE or FALSE
7 == 7
#generates TRUE wheres
6 == 7
#generates FALSE
```

2.2.8 aes (aesthetics in ggplot)

The `aes` is important for telling the ggplot what to plot. `aes` are the aesthetics of the plot that need to mapped to data. So the ggplot needs `data` and `mappings`.

The `ggplot` acronym is actually coming from the `grammar of graphics`, which is a book "The grammar of graphics" by Leland Wilkinson, and was used by Hadley Wickham to make the `ggplot` package in 2005.

A `ggplot` consists of: - data - aestehtic mappings (like x, y, shape, color etc) - geometric objects (like points, lines etc) - statistical transformations (stat_smooth) - scales - coordinate systems - themes and layouts - faceting

```
#ggplot basics with one geometric object "geom_point"
#and several aesthetics
  ggplot(data = mtcars,
```

```
        mapping = aes(x = mpg,
                      y = disp,
                      color = hp,
                      shape = as.factor(cyl))) +
  geom_point()
```

2.2.9 %in% (match operator)

This is handy to check and filter specific elements from a vector

```
my_groups <- c("50.000", "100.000", "150.000")

"50.000" %in% my_groups #generates TRUE

#and the other way around
my_groups %in% c("50.000", "100.000")

#this is usefull when filtering specific elements in a tibble
iris %>%
  filter(Species %in% c("setosa", "virginica"))
```

2.3 Practical tips

2.3.1 Running your code

Webr code in the browser can be run as a complete code block by clicking on the `Run code` button when the webr status is `Ready!`, right above the block.

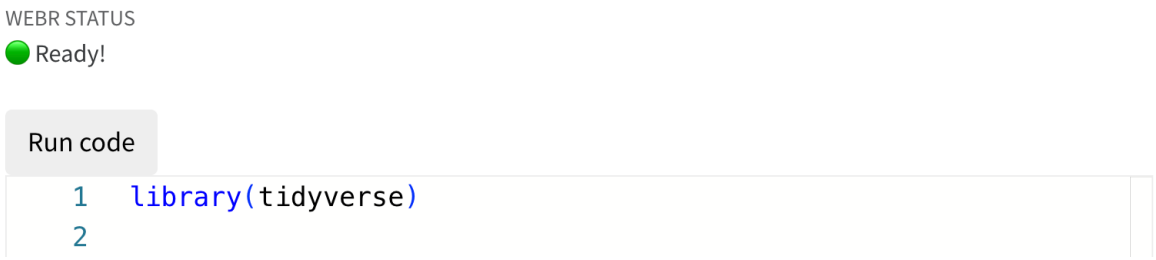

Figure 2.1: Screenshot of a code block that is ready to run

Another option is to select a line of code (or more lines) and press `command or ctrl enter`. This will execute only the line or lines that you have selected.

2.3.2 Simple troubleshooting your pipelines and ggplots

It happens that your code is not right away typed in perfectly, so you will get errors and warnings. It is good practice to break down your full code block or pipe into parts and observe after which line of code the code is not working properly.

2.4 ADVANCED EXERCISE

2.4.1 Building your data visualisation step by step

Let's take a built-in R dataset `USArrests`. We want to visualize how the relative number of murders in the state Massachusetts relates to the other states with the highest urban population in those state. In the dataset, the `murder` column represents the `number of murders per 100.000 residents`

```
USArrests

head(USArrests)

glimpse(USArrests)

#please note that the states are listed as rownames. The glimpse does not show the rownames!
```

Exercise x

Make a plot that addresses the above dataviz problem.

```
USArrests #%>%
  #......
  #......
  #......
  #......
  #ggplot.....
    #.......
    #etc
```

HINTS

Hints:
Do the following in your coding:

- `glimpse` at the data and look at the top5 rows using `head()`
- use `tibble::rownames_to_column()` to make a separate column called `states`
- clean the column names using `janitor::clean_names()`
- turn the datatable into a `tibble` using 'as_tibble'
- take only the the top states by using a filter on the urban population (take it higher than 74)
- plot the data using a `geom_col`
- label the x axis and not the y-axis
- highlight the massachusetts column using a separate `geom_col` layer, were you put a filter on the original data by using in the `geom_col` a call to `data = . %>% filter(str_detect(states, "Mass"))`. Also give this bar a red color.
- apply a nice theme so that there are only x axis grid lines and no lines for y and x axis.
- Also make sure that x-axis starts at zero
- Use the `forcats::refactor()` to sort the states on the y-axis from highest murder to the lowest murder rate.

Include all these aspects step by step.

Solution to Exercise x

```
#webr::install("forcats")

USArrests %>%
  tibble::rownames_to_column(var = "states") %>%
  janitor::clean_names() %>%
  as_tibble() %>%
  filter(urban_pop > 74) %>%
  ggplot(aes( x = murder,
              y = forcats::fct_reorder(states, murder)))+
    geom_col(fill = "grey70")+
    geom_col(data = . %>%
               filter(stringr::str_detect(states, "Mass")),
             fill = "red")+
    labs(y = "",
         x = "number of murders per 100.000 residents")+
    scale_x_continuous(expand = c(0,0))+
    theme_minimal(base_size = 18)+
    theme(panel.grid.major.y = element_blank())
```

3 Plotting cars

3.1 Learn and code

First, let's make a simple scatter plot. We'll use a famous dataset that is used in R a lot for educational purposes. This is the `mtcars` dataset. It stands for *"Motor Trend Car Road Tests"*. See parameter overview and documentation for info about the `mtcars` dataset. It is one of many datasets available from base R or tidyverse packages, so we can always call it without having to load it.

First, we will inspect the dataset. For this we will load the tidyverse:

```
library(tidyverse)
```

Once tidyverse is loaded via the `library` call, it is available in your current session in your browser, so you do not have to load it each time. Let's have a look at the full dataset:

```
mtcars
```

or

```
#if you get an error here,
# please load the library call to tidyverse
mtcars %>% glimpse()
```

or

```
mtcars %>% head()
```

or

```
mtcars %>% tail()
```

Let's select a small part of the data using `select` from the `dplyr` package:

```
mtcars %>%
    select(mpg, disp)
```

Next, make a simple plot with the miles per gallon (`mpg`) and displacement parameters (`disp`) in the `mtcars` dataset.

```
mtcars %>%
    select(mpg, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point(size = 4)
```

This is a very basic plot, without much formatting. Let's make it prettier!

Add color and bring in a third parameter:

```
mtcars %>%
    #added cyl to the selection here
    select(mpg, disp, cyl) %>%
    ggplot(aes(x = mpg,
               y = disp,
               color = cyl) #added color to the aesthetics here
               ) +
               geom_point(size = 4)
```

Here we need to have a look at data-types. The `cyl` parameter is numerical. `ggplot` automatically assumes we want a continuous scale for this. Instead the `cyl` is more of a categorical data type (there are either 4, 6 or 8 cylinders in each car) so we can explicitly make the `cyl` parameter categorical like this:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl))) +
        geom_point(size = 4)
```

If you want to have different colors you can use one of the many color palettes available:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl)))+
        geom_point(size = 4)+
  #I also manually changed the name of the legend here
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")
```

Apart from color you can change the shape of the datapoints:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl),
#please note to also add shape to the aesthetics here
               shape = as.factor(cyl))) +
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                           name = "cylinders")
```

`ggplot` can use different themes for your plots, and there are many many options to tweak your plots to the way you like. You can see some examples below:

Let's change titles:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
                color = as.factor(cyl),
                shape = as.factor(cyl)))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
```

```
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")
```

Change the plotting theme and base size of the elements:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
                color = as.factor(cyl),
                shape = as.factor(cyl)))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        theme_bw(base_size = 20)
```

Change the scaling of the axes. It is good practice to plot graphs from zero:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
                color = as.factor(cyl),
                shape = as.factor(cyl)))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(limits = c(0, NA),
                           expand = c(0,NA))+
        scale_y_continuous(limits = c(0, NA),
```

```
                            expand = c(0,NA))+
        theme_bw(base_size = 20)
```

Now the datapoints at the maxima of the axis are not completely visible so it would be nice that we have some more space:

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
                color = as.factor(cyl),
                shape = as.factor(cyl)))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
```

Now we have generated a nice visualisation of our data using `ggplot`. Please note that `ggplot` uses layers and we added each time a different layer of information to the ggplot. If you want you can go wild with `ggplot`. Please find a nice overview of visualisations using ggplot, tidy and R from Cedric Scherer. Also the underlying R code is available for those plots.

3.2 Exercises

3.2.1 Adding layers and changing the MTCARS plot

Exercise 1

Give the points in the ggplot some transparency (or opacity), so that individual points are better visible. TIP: use the `alpha` argument it should be a number from 0 to 1.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl),
               shape = as.factor(cyl)))+
        geom_point(size = 4,
                   #type your extra code here:

                    )+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
```

Solution to Exercise 1

Please note that the alpha we added is not part of an aesthetics (`aes`), meaning that the value of the alpha is not linked with a parameter in our data.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
                color = as.factor(cyl),
                shape = as.factor(cyl)))+
        geom_point(size = 4,
                   #type your extra code here:
                    alpha = 0.7
                    )+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                        name = "cylinders")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
```

Exercise 2

Add a layer that will generate a smooth linear regression line that shows the relation between `mpg` and `disp`. Use the `stat_smooth` command for this.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point(size = 4,
                   alpha = 0.7)+
        # enter code here

        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
```

Solution to Exercise 2

Please make sure that the ggplot is not separated into groups. If the data is grouped by `color` or `shape` a different regression line for each group will be generated.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point(size = 4,
                   alpha = 0.7)+
        # enter code here
        stat_smooth(geom = "line",
                    method = "lm",
                    formula = "y ~ x")+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
```

Exercise 3

Use the `facet_wrap` command to make three separate plots for each cylinder.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point(size = 4,
                   alpha = 0.7)+

        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)
        # enter code here
        #(and don't forget to at a plus to the last line)
```

Solution to Exercise 3

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp))+
        geom_point(size = 4,
                   alpha = 0.7)+
        labs(title = "My cool MTCARS plot",
                x = "miles per gallon",
                y = "dispension")+
        scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
        theme_bw(base_size = 20)+
        facet_wrap(~cyl)
```

3.2.2 Fixing common errors

Below is some code that is not working properly, because of coding semantics mistakes. Can you spot (and fix) the errors?

❗ Fix error 1

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp
               color = cyl))+
        geom_point(size = 4)
```

🔥 Solution to Error 1

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp, #the error was here
               color = cyl))+
        geom_point(size = 4)
```

Commas are often forgotten, but easily fixed. Within brackets arguments are separated with commas. R also generates an error that is helpful and can point you to the missing ,.

❗ Fix error 2

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = cyl)) %>%
        geom_point(size = 4)
```

Solution to Error 2

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = cyl))+ #the error was in this line
        geom_point(size = 4)
```

`ggplot` layers are added with a `+` not with the `pipe` term.

Fix error 3

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp
               color = cyl)) +
        geom_point(size = 4) +
```

Solution to Error 3

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = cyl))+
        geom_point(size = 4) #the error was in this line
```

Make sure that the end of a layer or line of code is not followed up with a `+` or `%>%`.

Fix error 4

Although R doesn't show you an error message, the code does not give you what you want. The plot should show the `cyl` parameter in different shapes, just like there are three different colors for each level of the `cyl` parameter.

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl)),
                shape = as.factor(cyl))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                           name = "cylinders")
```

Solution to Error 4

```
mtcars %>%
    select(mpg, cyl, disp) %>%
    ggplot(aes(x = mpg, y = disp,
               color = as.factor(cyl), #parenthesis error
               shape = as.factor(cyl)))+
        geom_point(size = 4)+
        scale_color_brewer(name = "cylinders",
                           palette = "Set2")+
        scale_shape(solid = TRUE,
                           name = "cylinders")
```

The `shape` argument should be included in the aesthetics (`aes`) part of the ggplot

Solution to Error 4 using chatGPT

Since chatGPT can also solve basic programming problems https://doi.org/10.1371/journal.pcbi.1011511, I tried solving Error 4 also with chatGPT. In my chatGPT session, the following solution was given:

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             color = as.factor(cyl),
             shape = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_shape_manual(
    name = "Cylinders",
    values = c(1, 2, 3), # Customize shape values as needed
    labels = c("4", "6", "8")
  )
```

chatGPT indeed put the `shape` in the `aes`, but also started making other changes. The remark that chatGPT made was: *"you should correct the syntax error and use scale_shape_manual() instead of scale_shape()."*... *"to customize the shape values and labels."*. I didn't really ask for that, and from there it went all downhill. Let's call that the "chatGPT loop of death". Let's see what happened.
Running the "improved" code showed that there was still no `fill`, so it did not improve in a way that I wanted.
Next I said *"but now the shapes don't have a fill"* and chatGPT replied:
"I see your concern. If you want to have both color and fill for the shapes, you can achieve this by mapping fill within the aes() call and using scale_fill_brewer() for the fill scale. Here's the modified code to achieve that:"

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             color = as.factor(cyl),
             shape = as.factor(cyl),
             fill = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_fill_brewer(name = "Cylinders", palette = "Set2") +
  scale_shape_manual(
    name = "Cylinders",
    values = c(1, 2, 3), # Customize shape values as needed
    labels = c("4", "6", "8")
  )
```

This again did not give fill unfortunately. Asking for a different solution: *"the fill is not there, do you have another solution?"*

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             color = as.factor(cyl),
             shape = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_fill_manual(
    name = "Cylinders",
    values = c("4" = "red", "6" = "blue", "8" = "green")
  ) +
  scale_shape_manual(
    name = "Cylinders",
    values = c(1, 2, 3), # Customize shape values as needed
    labels = c("4", "6", "8")
  )
```

Also did not give me fill.
Another attempt by chatGPT (still not working):

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  mutate(cyl_shape_fill = as.factor(cyl)) %>%
  ggplot(aes(x = mpg, y = disp,
             shape = cyl_shape_fill,
             color = cyl_shape_fill)) +
  geom_point(size = 4) +
  scale_shape_manual(
    name = "Cylinders",
    values = c("4" = 21, "6" = 22, "8" = 23),
    labels = c("4", "6", "8")
  ) +
  scale_color_manual(
    name = "Cylinders",
    values = c("4" = "red", "6" = "blue", "8" = "green")
  )
```

Another attempt by chatGPT (still not working):

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             shape = as.factor(cyl),
             color = as.factor(cyl))) +
  geom_point(size = 4) +
  geom_point(aes(fill = as.factor(cyl)), size = 2,
             show.legend = FALSE) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_fill_brewer(name = "Cylinders", palette = "Set2") +
  scale_shape_manual(
    name = "Cylinders",
    values = c(1, 2, 3), # Customize shape values as needed
    labels = c("4", "6", "8")
  )
```

Asking chatGPT to use `scale_shape` and `solid=TRUE` gives:

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             color = as.factor(cyl),
             shape = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_shape(solid = TRUE,
              name = "Cylinders",
              breaks = c("4", "6", "8"),
              labels = c("4", "6", "8"),
              values = c(15, 16, 17),
              guide = guide_legend(
                override.aes =
                    list(fill = c("red", "blue", "green")))
  )
```

…. but apparently chatGPT doesn't know how to use `scale_shape` (in the first instance), because it `scale_shape` cannot take `values` as an argument. So when I reply with the following *"I get the following error: Error: unused argument (values = c(15, 16, 17))"*, chatGPT goes back to use the 'scale_fill_manual'.

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             color = as.factor(cyl),
             shape = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(name = "Cylinders", palette = "Set2") +
  scale_shape_manual(
    name = "Cylinders",
    breaks = c("4", "6", "8"),
    labels = c("4", "6", "8"),
    values = c(15, 16, 17)
  ) +
  scale_fill_manual(
    name = "Cylinders",
    breaks = c("4", "6", "8"),
    labels = c("4", "6", "8"),
    values = c("red", "blue", "green")
  )
```

AND FINALLY we have a nice and simple fill with one legend…… but also lots of unnecessary code. So after a few exchanges to make the code more concise this is what came out of it:

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             shape = as.factor(cyl),
             color = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(palette = "Set2",
                     name = "Cylinders") +
  scale_shape_manual(name = "Cylinders",
                     values = c("4" = 15,
                         "6" = 16,
                         "8" = 17)) +
  scale_fill_brewer(palette = "Set2",
                     name = "Cylinders")
```

Affter asking to use `scale_shape` instead of `scale_shape_manual`, chatGPT generates:

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             shape = as.factor(cyl),
             color = as.factor(cyl))) +
  geom_point(size = 4, stroke = 1) +
  scale_color_brewer(palette = "Set2", name = "Cylinders") +
  scale_shape(solid = TRUE, name = "Cylinders")
```

This works nicely, but chatGPT introduces `stroke = 1`, which is not needed here, so again we have unnecessary code. So after I asked *"can I leave out the stroke argument?"* we get the easiest solution and exactly the same solution as I came up with myself **without chatGPT**.

```
mtcars %>%
  select(mpg, cyl, disp) %>%
  ggplot(aes(x = mpg, y = disp,
             shape = as.factor(cyl),
             color = as.factor(cyl))) +
  geom_point(size = 4) +
  scale_color_brewer(palette = "Set2", name = "Cylinders") +
  scale_shape(solid = TRUE, name = "Cylinders")
```

Please note, that when building the ggplot example, I did use google (...off course) to get some solutions, I liked the `scale_shape` and `solid=TRUE` solution that I found, because it made the code so concise and I don't like to type in `values` and `breaks` manually.
ChatGPT use in science and coing is just dipping the toe in the water. ChatGPT is likely to better not be used as knowledge database but instead as **"reasoning or infering agents"** https://www.nature.com/articles/s41591-023-02594-z. ChatGPT can produce false information, also described as **"hallucinations"** https://www.nature.com/articles/d41586-023-00816-5, which makes it difficult to use it for getting knowledge and facts. That said, it can be used to gain knowledge and learn better coding skills. Here is a nice *quick tips* paper from PLOS computational biology on how to *"harness the power of chatGPT"* https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1011319.

4 Plotting seahorse

Now, lets plot some Seahorse data. For this we need to import some of it into this session .The data is available from github.

It is data from PBMCs where we followed OCR and ECAR using Extracellular Flux analysis with the XFe96 over time and during that time we injected after three measurement phases FCCP, and after six measurement phases we injected Antimycin/Rotenone (AM/Rot). Much more context can be found in Scientific Reports Janssen et al..

```
library(tidyverse)

#set file source
root_srcfile <-
  "https://raw.githubusercontent.com/vcjdeboer/"
repository_srcfile <-
  "seahtrue/main/inst/extdata/"

#download file and rename to "VB.xlsx"
download.file(
  paste0(
    root_srcfile,
    repository_srcfile,
    "20191219_SciRep_PBMCs_donor_A.xlsx"),
  "VB.xlsx")

#read xlsx file
xf<-readxl::read_xlsx("VB.xlsx", sheet = "Rate")

xf %>% glimpse()
```

As you can see from the `glimpse`, the data table that we have now (called a `tibble` in tidy language), contains 7 columns; `Measurement`, `Well`, `Group`, `Time`, `OCR`, `ECAR`, `PER`. The data is allready nice and tidyly organized in the Rate sheet of the excel file that we have loaded. The file was generated in the Wave Agilent software and directly comes from exporting the Seahorse data to xlsx.

I prefer to use lower case column names without any spaces, so for these column names we have to turn them into lower case first. We use some easy functions from the `janitor` package for this.

```
xf %>% janitor::clean_names()
```

Next, we can start plotting data using `ggplot`. Let's introduce the `filter` command from `dplyr`. Whereas `select` selects columns, `filter` selects rows. So let's filter the rows for the group which is labeled "200.000" (200.000 cells/per well) and the "Background" group.

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  glimpse()
```

i The filter command

Filtering data is selecting the rows based on some arguments. You need some to understand some semantics here. For filtering based on multiple conditions we use `group %in% c("200.000", "Background")`, for filtering based on a single condition we can use `group == "200.000"`. The `%in%` operator is used to match two or more items.

```
1 %in% c(1, 2, 3, 4, 5) #is TRUE
```

```
[1] TRUE
```

```
# just like
1 == 1 #is TRUE
```

```
[1] TRUE
```

```
#the reverse is also possible
c(1,2,3,4,5) %in% 1
```

```
[1]  TRUE FALSE FALSE FALSE FALSE
```

```
#is TRUE FALSE FALSE FALSE FALSE FALSE
```

```
#Try the 1 = 1 here as well

1 = 1

#Remember:
#the = operator is reserved for assignment
#just like the <- operator
# == is used for comparison
```

```
#say that we have the following vector
#( = groups in experiment)
group <- c("Background", "50.0000",
           "100.000", "150.000",
           "200.000", "250.000",
           "300.000")

#we can do the same without typing the
#names by hand like this:
group <- xf %>%
          pull(Group) %>%
          unique()

#then

c("200.000", "Background") %in% group

#generates TRUE TRUE

group %in% c("200.000", "Background")

#generates:
#TRUE FALSE FALSE FALSE TRUE FALSE FALSE
```

Thus the `group %in% c("200.000", "Background")` statement in the filter function above tells which group items to use. For `200.000` there is match (`TRUE`), but for `100.000` there is not a match (it is `FALSE`).

Now that we know how to filter we can use the filtered data to make the ggplot.

```
xf %>%
  janitor::clean_names() %>%
```

```
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr))+
  geom_point()
```

That plot is not very informative. Let's make it prettier. First, add a line plot:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
  #the group command in aes is important for lines
  #ggplot wants to know how to connect dots
             group = well,
             color = group))+
    geom_point()+
    geom_line()
```

Next, change colors:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    scale_color_brewer(palette = "Set1")
```

Change theme and text size:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
```

```
    geom_point()+
    geom_line() +
    scale_color_brewer(palette = "Set1")+
    theme_bw(base_size = 16)
```

Add titles:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

This is a very nice plot now! It shows all OCR curves for each well for the `200.000` and the `background` groups. The information that is now not in the plot is which line matches to which well.

We could consider coloring each line, but there are too many wells so it will not look nice! *TODO: Change this in the above code `color = well` instead of `color = group`. * You will notice that there are not enough colors in the `brewer` palette `Set1`, so you go back to the default coloring by deleting the `scale_color_brewer` line as well. Use th `#` to comment out the line. * Now notice that the legend is huge and not completely visible, againg indicating that this is not the way to go

Instead, we can try to label the lines. We'll use the `geom_text` or `annotate` commands from `ggplot`.

> **i** `ggrepel`
>
> For using plot labels personal R projects, consider using `ggrepel`. Unfortunately, it is not available for webr, but it has the benefit of automatically preventing text overlap. *TODO: consider removing this note*

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
          # here are the labels using geom_text
     geom_text(data = . %>%
                    filter(time ==
                             max(time)) %>%
                    filter(ocr  ==
                             min(ocr)),
               aes(label = well),
               vjust = 2,
               hjust = 1)+
     geom_text(data = . %>%
                    filter(between(time, 15, 20))%>%
                    filter(ocr  == max(ocr)),
               aes(label = well),
               vjust = -0.3,
               hjust = 1)+
    geom_point()+
    geom_line() +
    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Although we now labeled lines that are at the minimum and maximum OCR, this is only useful for this one plot in these conditions. The position of the label is tweaked based on this specific plot, making this not such a quick solution to our problem.

Subsetting of data within the ggplot commands

In the above ggplot commands, we included the `geom_text`, but we only used a subset of the full data for this geom. We use the `.` (dot) operator to get the original data (so in our case the filtered data that went into the ggplot), and piped that into another two filters. Basically we do the following, but then within one layer of the ggplot:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  filter(time ==  max(time)) %>%
  filter(ocr  == min(ocr))

#and we also use the between function from dplyr
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  filter(between(time, 15, 20)) %>%
  filter(ocr  == max(ocr))
```

Thus here we are filtering all the way to getting only one row of the full dataset. The `well` name "C08" or "B08" is then given to the `label` argument of `geom_text`.

Let's do some more layout adjustments. Although the `theme_bw` gives a basic plotting layout, we often want to change the formatting. There are again great resources for this, for example this one: https://ggplot2.tidyverse.org/articles/faq-customising.html, but we explain the basics here. By giving options to the `theme` function we can change specific elements of a ggplot.

For example, if we want to change the text size of the axis title (or leave it blank), we give arguments to the `axis.title` options. Also please note the `rel(1.2)` argument which means `relative 1.2 times higher than base_size`. I think it is good practice to use the `rel` here instead of absolute numbers.

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr))+
  geom_point()+
  labs(x = "time (min)",
       y = "OCR (pmol/min)")+
  theme_bw(base_size= 16)+
  theme(
    axis.title.x = element_blank(),
    axis.title.y = element_text(size = rel(1.2))

  )
```

Change the rel `1.2` to `0.5` in the above code and see what happens.

Next, we change the grid lines:

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr))+
  geom_point()+
  labs(x = "time (min)",
      y = "OCR (pmol/min)")+
  theme_bw(base_size= 16)+
  theme(

   panel.grid.minor = element_line(color  = "red"),
   panel.grid.major.x = element_line(color  = "blue"),

  )
```

Next, we change the orientation of the x axis labels.

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr))+
  geom_point()+
  labs(x = "time (min)",
      y = "OCR (pmol/min)")+
  theme_bw(base_size= 16)+
  theme(
    axis.text.x = element_text(
      angle = 45,
      vjust = 1, # vertical alignment
      hjust = 1, # horizontal alignment
      size = rel(2))
  )
```

ggiraph

`ggiraph` is another interesting package. This brings in some nice interactivity into the plot. Since we are now working with the plot in a browser, this can be very handy. This can also be useful if we want to publish the plot as html and not a plain PDF. `ggiraph` is unfortunately also not available for wasm/webr since one dependent package is not available `uuid`, and I also can't get it to run via quarto. *TODO: consider removing this

note*

Exercise 1

Add three vertical lines to the plot. You can use the `geom_vline` command with xintercepts set at 15, 33 and 48; so that the the line is approximately at the injection time point. Also give it a shade of grey, eg. `grey40`.

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    # add code here


    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Solution to Exercise 1

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Exercise 2

Now add the injection labels. Use the `annotate` command and *TODO: something seems to be missing here*

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    # example annotate
    annotate("text", x = 0, y = 155,
             label = "init", color = "grey40",
             hjust = 1, vjust = -0.1,size = 4, angle = 90)+
    # add other annotates here

    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Solution to Exercise 2

```
xf %>%
  janitor::clean_names() %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(0,15,33,48),
               color = "grey40")+
    annotate("text", x = 0, y = 155,
             label = "init", color = "grey40",
            hjust = 1, vjust = -0.1,size = 4, angle = 90)+
    annotate("text", x = 15, y = 155,
             label = "fccp", color = "grey40",
            hjust = 1, vjust = -0.1,size = 4, angle = 90)+
    annotate("text", x = 33, y = 155,
             label = "am/rot", color = "grey40",
            hjust = 1, vjust = -0.1,size = 4, angle = 90)+
    annotate("text", x = 48, y = 155,
             label = "monensin", color = "grey40",
            hjust = 1, vjust = -0.1,size = 4, angle = 90)+
    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Exercise 3

Use the `facet_wrap` command to plot all groups (except background) in separate plots and in each plot show the wells. First, we will need to filter away the background data. Instead of selecting all groups we need it is better and easier to this filter our the background data using `filter(group != "Background")`. The `!=` means "is not", and it is the inverse of the `==` operator.
Next, add the `facet_wrap` command to the ggplot. I prefer to do that always at the bottom, so that I can easily see if a plot is wrapped.

```
xf %>%
  janitor::clean_names() %>%
  #change this line:
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
    #add facet wrap here
    #and do not forgot to add a "+" in previous line
```

Solution to Exercise 3

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(0,15,33,48),
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)+
    facet_wrap(~group)
```

Exercise 4

The plot in exercise 3 looks great already, but the order of the plots is important! We would like to see it go from low to high OCR. We can fix that using the `forcats` package commands. A nice and quick way to sort is based on the name of the group. It is important to realize that the `Group` column in the `XF` data are characters and not numbers. That is also the reason why it doesn't get sorted in the most natural way. It is sorted based on the first character, thus the "50.000" group comes last. If we would change the "group" column to `double` (that is a number format), it would sort better, but also your group name will change because it will recognize the `.` as a decimal operator. So it is better to leave the group names as they are and do it differently.
In comes `forcats`, you can re-level and reorder the crap out of your data in the ggplot! We often do the releveling at the point where you use your parameter, without making any changes the type of the columns. So that means you can use `~fct_reorder(group, group)` in the `facet_wrap` instead of only `~group`.
Please note that `fct_reorder` first argument is the parameter that you plot or need, and the second argument is the parameter that is used for sorting the data. In our case now that is the same, both are "group", but we also need to add something else. If we would do it like this there will be no difference from when ggplot takes facet_wrap only takes `~group`. Thus we can make the second argument into a number by using `as.double`.

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)+
    facet_wrap(~group)
    #change the facet_wrap to
    #sort by number using fct_reorder and as.double
```

Also, try what happens in the above code if you:

- only use `as.double` in the facet_wrap
- change the type of data to `double` for the group column

Solution to Exercise 4

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(0,15,33,48),
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)+
    facet_wrap(~fct_reorder(group, as.double(group)))
```

Exercise 5

Now that the `facet_wrap` is sorted nicely, we would also like to have the legend sorted nicely. Use the same `fct_reorder` trick to reorder the color legend.

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+ #change here
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
    #take solution from previous exercise
    #to have facet_wrap sorted here
```

If you didn't already change the title of the legend, do that as well. You can specify the name of the legend manually using the `name` argument in the `scale_color_brewer` command.

Solution to Exercise 5

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = fct_reorder(group, as.double(group))))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2",
                       name = "group")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)+
    facet_wrap(~fct_reorder(group, as.double(group)))
```

Exercise 6

Please change the `facet_wrap` command so that the y-axis is not fixed for all groups. Make the output so that each individual plot has its own y-axis scale.

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
    #take solution from previous
    #and adjust that facet_wrap
```

Solution to Exercise 6

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  ggplot(aes(x = time, y = ocr,
             group = well,
             color = fct_reorder(group, as.double(group))))+
    geom_point()+
    geom_line() +
    geom_vline(xintercept = c(15,33,48) ,
               color = "grey40")+
    scale_color_brewer(palette = "Set2",
                       name = "group")+
    labs(subtitle = "OCR of increasing cell densities",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)+
    facet_wrap(~fct_reorder(group, as.double(group)),
               scales = "free_y")
```

Exercise 7

Now, it is up to you to build a whole ggplot using the XF data. Instead of plotting time vs OCR, now plot cell density vs maximal capacity. For this you need to know some stuff.

1. we define maximal capacity as the OCR at measurement 4
2. we should filter out the "Background" group
3. we should convert the group names to numbers
4. we can also add the mean of all wells for each group by using: `stat_summary()`

```
xf %>%
  janitor::clean_names()
  #filters and ggplot here
```

Solution to Exercise 7

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  filter(measurement == 4) %>%
  ggplot(aes(x = as.double(group)*1000, y = ocr))+
    geom_point()+
    stat_summary(fun = "median",
      colour = "red",
        size = 16,
        shape = "-",
      geom = "point")+
    scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
    scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
    labs(subtitle =
           "Maximal capacity at different cell densities",
         x = "cell density (#cells)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Exercise 8

The previous plot showed the data from individual wells as well as the median for that group. You can also calculate the `median` before plotting using the `dplyr summarize` command. You can find `summarize` info here: https://dplyr.tidyverse.org/reference/summarise.html

```
xf %>%
  janitor::clean_names()
```

Solution to Exercise 8

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  filter(measurement == 4) %>%
  summarize(median = median(ocr), .by = group)%>%
  ggplot(aes(x = as.double(group)*1000, y = median))+
  geom_point()+
  geom_line()+
  labs(x = "cell density",
       y = "OCR (pmol/min")+
  theme_bw(base_size = 16)
```

Exercise 9

We can also perform a linear regression on the maximal capacity at different densities. For this we can use the `geom_smooth` command. The arguments should be `method = "lm"` and `formula = y~x`.

```
xf %>%
  janitor::clean_names()
  #use your code from exercise 7
```

Solution to Exercise 9

```
xf %>%
  janitor::clean_names() %>%
  filter(group != "Background") %>%
  filter(measurement == 4) %>%
  #filter(str_detect(well, "A|H")) %>%
  ggplot(aes(x = as.double(group)*1000, y = ocr))+
    geom_point()+
    stat_summary(fun = "median",
      colour = "red",
        size = 16,
        shape = "-",
      geom = "point")+
    geom_smooth(method = "lm",
                formula = y~x)+
    scale_x_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
    scale_y_continuous(
              limits = c(0, NA),
              expand = expansion(mult = c(0, 0.1)))+
    labs(subtitle =
           "Maximal capacity at different cell densities",
         x = "cell density (#cells per well)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Observe also what the difference is when only using the data from rows A and H. You can uncomment the line in the above code. Please note I use the very useful `str_detect` function for this from the `stringr` package that is also part of the `tidyverse`.

Exercise 10

Next, you can decide yourself what you want to plot. Have a `glimpse` at the data and think of another important visualisation that you want to make using all the tools that you have learned so far, or the tools that you found on the internet.

```
xf %>%
  janitor::clean_names() %>%
  glimpse()

xf %>%
  janitor::clean_names()
```

5 Summary

5.1 What did we learn?

5.1.1 Ditching point-and-click

- The benefits of using R over point-and-click software for data analysis in biological and biomedical sciences are that:
  - it is open-source,
  - it has a wide and diverse community with a huge number of resources
  - it is relatively easy to learn, and
  - it offers very well suited workflows for doing reproducible and responsible data analysis
- The tidyverse offers advantages over base R. It offers an intuitive way of coding with functional names and tidy data handling and coding in mind
- R in the browser offers easy access to R without installing software

5.1.2 Plotting `mtcars`

- General R coding and execution of code
- How to look at data tables: `head`, `tail`, `glimpse`
- The pipe operator `%>%` or `|>`
- Making factorial data using `as.factor`
- the `dplyr` function `select`
- basic `ggplot` functions using `aes` aesthetics and geoms such as `geom_point`
- adding `color` and `shape` and using `scale_brewer_manual` and `scale_shape`
- improving layout; `theme_bw`, `base_size` and `labs`
- using chatGPT for coding improvements

5.1.3 Plotting Seahorse data

- Loading data and working with typical Seahorse data
- Using `janitor` `clean_names`
- Using the `dplyr` function `filter`

- Using the `%in%` operator
- Changing the layout of ggplots usine `theme` elements and arguments.
- Adding text to ggplot using `geom_text` and `annotate`
- Adding lines to ggplot using `geom_vline`
- Nesting pipes in ggplot function for subsetting data
- Using `facet_wrap` to make multiple similar plots from one datatable
- Using the `forcats fct_reorder` function
- Changing data formats to numbers using `as.double`
- Using the dplyr `summarize` function
- Using `stat_summary` to compute means or medians in ggplots
- Using `geom_smooth` to make regression lines

5.2 What we did not learn?

- Base R functions and how to address data in base R, eg `xf$OCR[xf$Group == "Background]` and `xf$Well[10]`
- Other important tidyverse functions, like `pivot_wider`, `pivot_longer`,
- More complicated functions like the `map` function from the `purrr` package
- Other simple ggplot geoms, like `geom_bar`, `geom_boxplot`, `geom_density`
- How to save images and plots for using them in other software

5.3 External resources

- Tutorials
  - R for reproducible scientific analysis is a great introductory material. It is free, easy to follow and **quick to learn and apply**. You can skip the very first section on RStudio if you want.
  - The starter guide for transitioning your Python projects to R can be very useful to get quickstarted in R if you are familiar with Python.
  - R packaging shows you how to write your own R packages.
- Longer reads
  - R for data science is a really good book about R, with a focus on tidyverse. It is available for free.
  - Advanced R is another pretty good book for those who want to go even deeper into the language. Also available for free.

Part II

Seahtrue basics - swim underwater

Now we will go for a bit more than just swimming in R code. We will go underwater and swim in shallow water to look at those seahorses. In this section we will introduce functions from the `seahtrue` package and explore the output of `seahtrue` functions.

6 Seahtrue functions

6.1 Functions

First, let's see what a `function` is in R. In the previous section, we used functions that changed our data, eg. `filter`, `select` and `clean_names`. Functions are just a bunch of code lines using any code and other functions you want to accomplish a task. Here are some formal definitions:

> Functions are "self contained" modules of code that accomplish a specific task. Functions usually take in some sort of data structure (value, vector, dataframe etc.), process it, and return a result. link

> A function in R is an object containing multiple interrelated statements that are run together in a predefined order every time the function is called. link

Functions take `arguments`, these are used as input for your function.

```
library(tidyverse)

#make the function
change_mtcars_cyl_to_x <- function(x){
  mtcars %>%
    mutate(cyl = x)

}

#call the function
change_mtcars_cyl_to_x(8)
```

Please note that it is good practice to use verbs in function names and address in the name what a function is doing. In our case we define a function `change_mtcars_cyl_to_x` because this is exactly what this function is doing.

Exercise 1

The `change_mtcars_cyl_to_x` function in the above code is a nonsensical function, because you never want to change a column to one specific value. Let alone a specific column named `cyl`. Also, you don't have to write a whole separate function for this, you can also directly use the `mutate` function from `dplyr`. Write the code without using the `change_mtcars_cyl_to_x` function, but achieve the same result.

```
# use mtcars and mutate from dplyr
```

Solution to Exercise 1

```
mtcars %>%
  mutate(cyl = 8)
```

Please note that you can change the data in a column to anything you want. R is very very flexible in datatypes (compared to other languages). So if you would do this:

```
mtcars %>%
  mutate(cyl = "eight")
```

that is also fine.
The data types are given when you `glimpse` the data.

```
mtcars %>% glimpse()
```

You see that all columns are `<dbl>` which stands for `double`, which is a numeric data type. `integer` is another common numerical datatype.
When you replace the `cyl` column data with `"eight"`, which is of the `character` type, the data type will change.

```
mtcars %>%
  mutate(cyl = "eight") %>%
  glimpse()
```

This is all fine in R. **Important to note** though is that a column can have only one data type. In Excel you can define each cell a different data type, but in R that is not possible. So it is either a column of type `character` or `double` in our case.

Now let's extend the function a bit to have two arguments:

```
#make the function
change_df_cyl_to_x <- function(df, x){

  df %>%
    mutate(cyl = x)

}

#call the function
change_df_cyl_to_x(mtcars, 7)

#or call with pipe
mtcars %>% change_df_cyl_to_x(7)

#you can also see this as:
mtcars %>% change_df_cyl_to_x(., 7)
```

Although the function is a bit more general, because we can now also input the tibble that we want to change, it is still not very useful in practice. A single `mutate` function is preferred to be used here. On the other hand it is an easy example to demonstrate what a function is and how it works.

6.2 Seahtrue read data function

Now let's start with the functions from the `seahtrue` package. Since `seahtrue` is not available for webr, we need to load in the functions manually. The first thing we do is to read data from the excel file that is generated using the Wave software. In the previous section we only loaded in one sheet of that datafile `Rate`, but the `seahtrue` package takes all data and organizes it nicely (and tidyly) into a `nested tibble`.

One of the functions is the `get_xf_raw`. It reads the Raw sheet from the excel file.

```
get_xf_raw <- function(fileName){

  xf_raw <- readxl::read_excel(fileName, sheet = "Raw")

}
```

The argument `fileName` and its location is important. If we work with data input for your scripts, you need to be precise where your files are located. On windows and mac computers and with cloud services and web apps, it can get confusion what this exact location is of your files, either locally or on network or cloud. Sometimes they are on the desktop or in a documents folder, or they can live on a network drive. Properly addressing these files can be difficult because the `full path` is not always known. It is often recommended to put data files in the Rstudio project folder that you work with, so that you can work with relative paths from your project root directory. This is another example of good practice.

Using webr/wasm we do a similar thing, we download the file to our local drives. On my computer when I download a file it goes into the `/home/web_user/` directory. Apparently this is my working directory in my webr/wasm sessions. Since it is the working directory, everything that is in there is directly accessible with only the filename. You don't need a full path name like `C:\Users\MyName\Desktop\R\projects\blabla\datafolder\data\`. So for the `get_xf_raw` function to work we first download the file into our session working directory and then we call the `get_xf_raw` function.

```
library(tidyverse)

#set file source
root_srcfile <-
  "https://raw.githubusercontent.com/vcjdeboer/"
repository_srcfile <-
  "seahtrue/main/inst/extdata/"

#download file and rename to "VB.xlsx"
download.file(
  paste0(
    root_srcfile,
    repository_srcfile,
    "20191219_SciRep_PBMCs_donor_A.xlsx"),
  "VB.xlsx")

#define the function
get_xf_raw <- function(fileName){

  xf_raw <- readxl::read_xlsx(fileName, sheet = "Raw")

}

#set the file name variable
fileName <- "VB.xlsx"
```

```
#read xlsx file
xf<-get_xf_raw(fileName)

#glimpse at the xf tibble
xf %>% glimpse()
```

Please note that we also tell our session what the `get_xf_raw` function is here. Basically, we assign the code lines to `get_xf_raw`, so when we call `get_xf_raw` with its argument, these lines of code are run.

> **i** using functions from packages
>
> In the `get_xf_raw` function we call the `read_xlsx` function. However, we also include the package from which the function is from, like this `readxl::read_xlsx`. This has two advantages. First, it is good practice to show where your function comes from, because sometimes a function name is used in mulitple different packages. For example, the `filter` function we often use is from `dplyr`, but the `stats` package also uses the `filter` function but then in a slightly different way. Second, when using the `package::function` annotation you don't have to load the package using the `library` command.
> In other languages, such as python, you are required to also include the library, when calling a function from that library https://www.rebeccabarter.com/blog/2023-09-11-from_r_to_python.

Apart from reading the `Raw` data sheet there are a couple more functions to read the other data and meta info.

```
#raw data
get_xf_raw()

#rate data
get_xf_rate()

#normalization data
get_xf_norm()

#buffer factors
get_xf_buffer()

#injection info
get_xf_inj()

#pH calibration data
```

```
get_xf_pHcal()

#O2 calibration data
get_xfO2cal()

#flagged wells
get_xf_flagged()

#assay info
get_xf_assayinfo()
```

Furthermore, there is a function that combines all functions as above and outputs them in a list:

```
#raw data
read_xf_plate()
```

The input argument for all is the filename or path of the input xlsx data file.

6.3 Seahtrue preprocess data function

Following reading the data, the data needs to be processed to a tidy format so that it can be easily used for downstream processing.

For example, there is a function which changes the columns from the input file data into names without capitals and spaces. The `clean_names` from the janitor package can also be used, but in this case we wanted to be a bit more precise on what the names should be.

```
rename_columns <- function(xf_raw_pr) {

  # change column names into terms without spaces
  colnames(xf_raw_pr) <- c(
    "measurement", "tick", "well", "group",
    "time", "temp_well", "temp_env", "O2_isvalid", "O2_mmHg",
    "O2_light", "O2_dark", "O2ref_light", "O2ref_dark",
    "O2_em_corr", "pH_isvalid", "pH", "pH_light", "pH_dark",
    "pHref_light",
    "pHref_dark", "pH_em_corr", "interval"
  )
```

```
  return(xf_raw_pr)
}
```

The next preprocessing function takes the `timestamp` (colnumn name is now `time`) from the `Raw` data sheet and converts the `timestamp` into minutes and seconds. This function has some more plines of code, but all it does is to add three columns to the tibble: `totalMinutes`, `minutes` and `timescale`. I used `timescale` here to make sure that I can recognize it as different from the `time` column.

```
convert_timestamp <- function(xf_raw_pr) {

  # first make sure that the data is sorted correctly
  xf_raw_pr <- dplyr::arrange(xf_raw_pr, tick, well)

  # add three columns to df (totalMinutes, minutes and time) by converting the timestamp into
  xf_raw_pr$time <- as.character((xf_raw_pr$time))
  times <- strsplit(xf_raw_pr$time, ":")
  xf_raw_pr$totalMinutes <- sapply(times, function(x) {
    x <- as.numeric(x)
    x[1] * 60 + x[2] + x[3] / 60
  })
  xf_raw_pr$minutes <- xf_raw_pr$totalMinutes - xf_raw_pr$totalMinutes[1] # first row needs t
  xf_raw_pr$timescale <- round(xf_raw_pr$minutes * 60)

  return(xf_raw_pr)
}
```

All other preprocessing steps and functions can be looked up in the `preprocess_xfplate.R` file on github https://github.com/vcjdeboer/seahtrue/blob/develop-gerwin/R/preprocess_xfplate.R. Combined the `preprocess_xfplate` function takes the output of the `read_xfplate` function and outputs all data in a nice data table consisting of a bunch of nested tibbles.

The `preprocess_xfplate` and `read_xfplate` functions are combined in the `run_seahtrue` function. The `seahtrue` has some extensive unit testing, user interaction, and input testing build-in using the `testthat`, `cli`, `logger` and `validate`.

The basic read and prepocess function looks like this.

```
run_seahtrue() <- function(filepath_seahorse){

  filepath %>%
    read_xfplate() %>%
```

```
    preprocess_xfplate()
}
```

In the next section we will explore the output of the `run_seahtrue` function.

Exercise 2

The `rename_columns` function could have also been written using `clean_names` from the `janitor` package. This would have been likely faster to implement. Replace a `clean_names` code that does the same as the `rename_columns` function

```
#
```

Solution to Exercise 2

Exercise 3

Do the same for the `convert_timestamp` function. Use the `lubridate` package to write a simpler code

```
#
```

Solution to Exercise 3

7 Seahtrue outputs

7.1 Nested tibbles

The data output format of the `run_seahtrue` function is a list of lists. List of lists is also called nesting of data. The advantage of this is that the data is properly organized, but also easily accessible. Here is an example that I took from a `tidyr` vignette https://tidyr.tidyverse.org/articles/nest.html.

```
library(tidyverse)

mtcars %>%
  nest(.by = cyl)
```

You can see that the the data is now nicely organized by the `cylinder` parameter. Since there are only 3 different values for the cyl in the mtcars dataset, there are now three rows and two columns, one column has the `cyl` parameter all other data is nested into a `data` column.

> **i** .by vs group_by
>
> In one of the latest releases of the `tidyverse` the use of `.by` was introduced. Previously we used the `group_by` to tell R how to organize the data. The grouping of data remains attached to the data tibble, which sometimes could result in unintentional things to happen, when you forgot that the tibble was grouped. The `group_by` can be undone with the `ungroup` command.
> With the `.by` the grouping is only apparent while using the function in which you use it as argument. `group_by` and `.by` are doing similar things so they can be used both.Let's have a look at how they work:

```
#first do a complete summarize
mtcars %>%
  summarize(meamn = mean(disp))


#second only summarize the disp for each cyl
mtcars %>%
  summarize(mean = mean(disp), .by = cyl)

#alternatively you can use
mtcars %>%
  group_by(cyl)%>%
  summarize(mean = mean(disp))
```

If you `glimpse` the results of the two ways of using grouping above you will see that `group_by` is doing stuff to your data, that you might not want. In this case it turns the `mtcars` dataframe into a tibble, whereas the result of the `.by` in the `summarize` function is still a dataframe. Although it might not really matter whether your data is a tibble or dataframe, it shows that `group_by` is a bit more invasive on your data.

You can use `pluck` to get to the nested `data`. Basically you just pluck a part of the data out of the full dataset.

```
mtcars %>%
  nest(.by = cyl) %>%
  pluck("data", 1)
```

Please note that we use here `"data"` instead of `data`. It can be confusing when to use the `""` or not. For example, with the `pull` function which takes one full column out of a tibble, you are not using `""`.

Also, `pluck` uses indexing for retrieving its components, it is not possible to directly get the element that belongs to `cyl == 3` for example. You would need to `filter` first on that parameter and then `pluck` the first row of data.

7.2 The purrr map function

The cool thing about a nested tibble is that you can quickly perform stuff on each nested tibble. A really good introduction to this is described in this blog post by Rebecca Barter

https://www.rebeccabarter.com/blog/2019-08-19_purrr. You can map a function on each item from that row.

```
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model =
           map(data, function(df) lm(mpg ~ wt, data = df)))
```

You see that a new column is generated named `model`, if you `pluck` the one of the models, you can see the typical output of the linear model (`lm`) function. For each cylinder now you creates a linear model!

```
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model =
           map(data,
               function(df) lm(mpg ~ wt, data = df))) %>%
  pluck("model", 1)
```

The semantics and how to use the `map` function is nicely explained in the blog post that was referenced here above. But some more considerations here:

```
#this is the original form
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model =
           map(data, function(df) lm(mpg ~ wt, data = df)))

#some of the parts are left out
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model =
           map(.x = data,
               .f = function(df) lm(mpg ~ wt, data = df)))

#this makes it a bit more clear
#now .x is the data and .f is the function
```

```
#also you can change the function syntax
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model = map(.x = data,
                     .f = ~ lm(mpg ~ wt, data = .x)))

#now function was replaced with ~
#and the df was replaced with .x

#this also works . instead of .x
mtcars %>%
  nest(.by = cyl) %>%
  mutate(model = map(.x = data,
                     .f = ~ lm(mpg ~ wt, data = .)))
```

Another good resource for the purrr map function is https://dcl-prog.stanford.edu/purrr-basics.html. `map` has many more forms and ways to use, which are summarized in its cheat sheet https://github.com/rstudio/cheatsheets/blob/main/purrr.pdf.

7.3 The seahtrue ouput

Now go and have a look at the `run_seahtrue` output.

```
library(tidyverse)

root_srcfile <-
  "https://raw.githubusercontent.com/vcjdeboer/"
repository_srcfile <-
  "seahtrue/develop-gerwin/data/"

download.file(
  paste0(
    root_srcfile,
    repository_srcfile,
    "seahtrue_output_donor_A.rda"),
  "seahtrue_output_donor_A.rda")

load("seahtrue_output_donor_A.rda")

seahtrue_output_donor_A %>% glimpse()
```

Also pluck some of the data

```
# get the original input filename and location
seahtrue_output_donor_A %>%
  pluck("filepath_seahorse",1)

# get the injection info
seahtrue_output_donor_A %>%
  pluck("injection_info",1)

# get the date when exp was run
seahtrue_output_donor_A %>%
  pluck("date",1)
```

Some data are simple character strings, like the `date` column, whereas others are large tables like the `raw_data` column

With this loaded data (`seahtrue_output_donor_A`) you can now do similar plotting as in the `plotting seahorse` chapter. For this we only have to `pluck` the `rate_data` out of the data set. Be carefull that we preprocessed the data and we have other column names now so first `glimpse` the data.

```
seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  glimpse()

#or get only the column names
seahtrue_output_donor_A %>%
  colnames()
```

You will see that the column names are labeled with `wave`, in this way we can distinguish for example the time column in the `raw_data` tibble from the `time_wave` column in the `rate_data` tibble. Also, please notice that we have `OCR_wave_bc` and `OCR_wave`. This distinctino is made because we can have OCR data that is background corrected or not. When clicking on the background slider in the Wave software from Agilent, the OCR data will be changed to non background corrected. If at this point the data is exported the xlsx input file is not background corrected. In the `seahtrue` this will show up as `OCR_wave`. Typically however the data is background corrected, so we most of the time have `OCR_wave_bc`.

> **time and time again**
>
> Since rate is an aggregate of mulitple O2 or pH readings, also the definition of the timing of each measurement is different between the `rate_data` and the `raw_data`. Therefore in the `seahtrue` package both times are labeled differently. For the `rate_table` we labeled it with `time_wave` and for the `raw_data` we labeled it with `timescale`. And again, we used `timescale` to distinguish it from the `time` in the original input file.

Please note if we want to plot the OCR vs time, we have to use the `OCR_wave_bc` vs `time_wave` in our ggplot aesthetics.

It is good practice to have a quick look at how the groups were named in the experiment. We can use the `pull(group)` and `unique()` commands for this:

```
#first have a look at what groups we have
seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  pull(group) %>% unique()
```

Next, take some of the groups and plot them in a ggplot:

```
seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  filter(group %in% c("200.000", "Background")) %>%
  ggplot(aes(x = time_wave, y = OCR_wave_bc,
             group = well,
             color = group))+
    geom_point()+
    geom_line() +
    scale_color_brewer(palette = "Set1")+
    labs(subtitle = "200.000 cells per well vs Background",
         x = "time (minutes)",
         y = "OCR (pmol/min)")+
    theme_bw(base_size = 16)
```

Great, this looks exactly the same as the plot we generated using the data from the downloaded excel file in the “plotting seahorse” chapter.

8 Summary

8.1 What did we learn?

8.1.1 Functions

- Definitions of functions
- Function argument
- How to build and execute functions
- How to use the `dplyr` `mutate` function
- Some basic understanding of the `seahtrue` `read_xfplate` functions
- Some of the `seahtrue` `preprocess_xfplate` functions
- The `run_seahtrue` function

8.1.2 Outputs

- What nested tibbles are, and why they are usefull
- How to generate nested tibbles
- How to use the `purrr` `map` function on nested tibbles
- Get familiar with the semantics of the `map` function
- The difference between `.by` and `group_by`
- How to isolate or `pluck` data from a larger dataset
- Get familiar with the `seahtrue` output nested tibble
- The different parameters for `time` in a seahorse dataset
- How to access the data in `seahtrue` output and use it in ggplot

8.2 What we did not learn?

- How to run seahtrue on our own data
- How all the assertions and input checking in the `seahtrue` package are implemented
- More elaborate use cases of the `map` function from the `purrr` package, like `map2` or `map_dbl`

Part III

Advanced Seahtrue - Diving deeper

In the previous chapters we learned R and got familiar with the `seahtrue` package and its output. Here we will explore what is possible with the seahtrue data in R. We will go through making some nice visualisations of the raw data and rate data. Also we will show how to combine multiple experiments into one tibble and visualize the data.

9 Seahorse Analysis

```
print("webr is working!")
```

9.1 Data

The experimental data is derived from a paper we published a couple of years ago. Please check-out the paper where this data was published at:

Janssen et al. (2021) Scientific Reports

Exercise x1

Look up in the paper: - what type of cells have been used? - what compounds were injected during the assays?

Solution to Exercise x1

For most of the experiments PBMCs have been used. For some experiments RAW264.7 cells were used or monocytes. The dataset we use in this tutorial was an experiment with PBMCs.
We used injections of FCCP, AM/rot, and monensin. Also Hoechst was included, but this was only used for nuclei staining

Let's load the data first. We use the `revive_xfplate()` function form the seahtrue package and the excel file that is included with the package.

```
library(tidyverse)

seahtrue_output_donor_A <-
  system.file("extdata",
     "20191219_SciRep_PBMCs_donor_A.xlsx",
     package = "seahtrue") %>%
  revive_xfplate()
```

Exercise x2

Explore the data that we just loaded.

```
seahtrue_output_donor_A %>%
  glimpse()
```

What is the structure of `seahtrue_output_donor_A`?

```
seahtrue_output_donor_A %>%
  pluck("rate_data", 1) %>%
  glimpse()
```

```
seahtrue_output_donor_A %>%
  pluck("raw_data", 1) %>%
  head()
```

And what is the structure of the `rate_data` and `raw_data` tibbles?

Solution to Exercise x2

The data is a nested tibble. The nested `rate_data` tibble contains the ocr and ecar data, with for each `well` the measurements and group annotation and the `OCR_wave_bc` and `ECAR_wave_bc` data columns "…._bc" in the columns names means the data is allready background corrected (bc)). The `raw_data` contains the O2 and pH readings in the experiment, those are used to calculate the OCR and ECAR).

9.2 Plate map

To dive a bit deeper into a single `seahtrue` experiment, we will first generate an overview of what the experimental set-up was.

Explore the experimental set-up. Remember that a Seahorse experiment is a 96-well plate based experiment, so we will see 96 wells in the plate map.

```
seahtrue_output_donor_A %>%
  pluck("raw_data", 1) %>%
  seahtrue::sketch_plate(reorder_legend = TRUE)
```

Exercise x3

In the plate map, you see that the corners are colored dark brown The background wells do not contain cells. What is the purpose of these background wells?

Solution to Exercise x3

There is drift in O2 and pH signals even when there is no sample (cells) in a well. This signal should be substracted from the wells that do contain samples.

Exercise x4

What do the numbers 50.000 to 300.000 in the legend of the plate map?

Solution to Exercise x4

These are the number of cells that have been plated in each well, so 50.000 cells/well to 300.000 cells/well

Exercise x5

What is the reason why we performed an experiment like this?

Solution to Exercise x5

In this way, we can check the linearity of the signals and determine the best plating density for our following experiments where we want to compare different conditions or interventions.

9.3 Oxygen consumption rate (OCR)

Now we will plot the OCRs for this experiment.

```
#before running this, make the seahtrue_output_donor_A
#is assigned in the above code

seahtrue_output_donor_A %>%
  purrr::pluck("rate_data", 1) |>
  sketch_rate(
```

```
        param = "OCR",
        normalize = FALSE,
        take_group_mean = TRUE,
        reorder_legend = TRUE
    )
```

Exercise x6

If we assume that the basal OCR should not be lower than 20 pmol/min (below this value we reach the detection limit), which cell densities are suitable for a Seahorse experiment in this case.

Solution to Exercise x6

from 150.000 cells/well

Exercise x7

Now check out the parameters that we can set for the `sketch_rate()` function by using the below code:

```
?seahtrue::sketch_rate()
```

plot the ECAR using this `sketch_rate()` functions. What do you observe after monensin is injected in the ECAR plot? Look up online how monensin acts on cells why it is doing this with ECAR. What did we observe in OCR? Can you explain that?

```
seahtrue_output_donor_A %>%
  purrr::pluck("rate_data", 1) #.....
  #.....
  #.....
```

Solution to Exercise x7

```
seahtrue_output_donor_A %>%
  purrr::pluck("rate_data", 1) |>
  sketch_rate(
         param = "ECAR",
         normalize = FALSE,
         take_group_mean = FALSE,
         reorder_legend = TRUE
     )
```

We see that after monensin injection, ECAR increases. This is because glycolysis is increased because cell membrane transporters are activated that increase ATP demand. OCR does not change much, because the mitochondrial activity is allready blocked by AM/rot

9.4 Plotting basal and maximal respiration

Pluck the `injection_info` table from the `seahtrue_output_donor_A` dataset to see what how the injections were defined in the experimental set-up before running the seahorse.

```
seahtrue_output_donor_A %>%
  pluck("injection_info",1)
```

This is not a typical mito-stress test experiment, where we inject oligomycin, FCCP and antimycinA/rotenone sequentially. Instead we inject only FCCP and antimycinA/rotenone.

To get the maximal and basal respiration out of the ocr rate table, we need to do some calculations. We first make some assumptions and defintions:

> We call each interval between two injections or between start and an injection or between an injection and the end a `phase`
>
> Each phase has a unique name that is named after the injection that was last. The first phase after the start is called `init_ocr` and we also typically have the phases `om_ocr`, `fccp_ocr` and `amrot_ocr`. Phases are marked with either `_ocr` or `_ecar`, because these are distinct parameters.
>
> To calculate respiration parameters, like basal respiration (= basal_ocr), we define the following:

- basal_ocr = init_ocr - amrot_ocr.
- max_ocr = fccp_ocr - amrot_ocr
- spare_ocr = fccp_ocr - init_ocr
- proton_leak = om_ocr = amrot_ocr
- atp_linked = init_ocr - om_ocr

We also use indices to have relative parameters:

- spare_ocr_index = (spare_ocr / basal_ocr)*100
- basal_ocr_index = (basal_ocr / max_ocr)*100
- leak_index = (proton_leak / basal_ocr)*100
- coupling_index = (atp_linked / basal_ocr)*100) %>%

Another important assumption is that we are not using average values to represent each phase, but intead we use a specific measurement. The reason for this is that we assume that for all phases, except FCCP, three measurements are needed in time to get to steady-state. For FCCP injection, we assume that it reaches steady-state fast, or at least its maximal ocr, so we take the first measurement after injection as the measurement representing the FCCP phase.

Let's now put that into R code. We call the type of experiment we did in this dataset a `maximal capacity` (`maxcap`) test.

We also injected monensin, which can maximize ECAR, but we don't need it for OCR calculations.

```
# first define which timepoints are what
param_set_maxcap_ocr <- c(init_ocr = "m3",
                         fccp_ocr = "m4",
                         amrot_ocr = "m9",
                         mon_ocr = "m12"
                        )

#next do some pivoting, renaming and selecting
seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  select(well,
         measurement,
         group,
         ocr = OCR_wave_bc) %>%
  pivot_wider(names_from = measurement,
              names_prefix = "m",
              values_from = ocr) %>%
```

```
  rename(all_of(param_set_maxcap_ocr)) %>%
  select(contains(c("wel"," group", "ocr")))
```

Now for each well we have the parameters related to the phases that we defined in the parameter set.

Next we want to calculate the respiration parameters and indices.

```
seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  select(well,
         measurement,
         group,
         ocr = OCR_wave_bc) %>%
  pivot_wider(names_from = measurement,
              names_prefix = "m",
              values_from = ocr) %>%
  rename(all_of(param_set_maxcap_ocr)) %>%
  select(contains(c("well","group", "ocr"))) %>%
  #piped here to the mutate
  mutate(non_mito_ocr = amrot_ocr,
         basal_ocr = init_ocr - non_mito_ocr,
         max_ocr = fccp_ocr - amrot_ocr,
         spare_ocr = max_ocr - basal_ocr,
         spare_ocr_index = (spare_ocr / max_ocr)*100,
         basal_ocr_index = (basal_ocr/max_ocr)*100)
```

With this data, we can plot our typical basal and maximal bar/scatter plots that we see in papers, presentations and theses.

```
param_set_maxcap_ocr <- c(
  init_ocr = "m3",
  fccp_ocr = "m4",
  amrot_ocr = "m9",
  mon_ocr = "m12"
)

plot_df <-
  seahtrue_output_donor_A %>%
  pluck("rate_data", 1) %>%
  select(well, measurement, group, ocr = OCR_wave_bc) %>%
```

```
  pivot_wider(
    names_from = measurement,
    names_prefix = "m",
    values_from = ocr
  ) %>%
  rename(all_of(param_set_maxcap_ocr)) %>%
  select(well, group, init_ocr, fccp_ocr, amrot_ocr, mon_ocr) %>%
  mutate(
    non_mito_ocr = amrot_ocr,
    basal_ocr = init_ocr - non_mito_ocr,
    max_ocr = fccp_ocr - amrot_ocr,
    spare_ocr = max_ocr - basal_ocr,
    spare_ocr_index = (spare_ocr / max_ocr) * 100,
    basal_ocr_index = (basal_ocr / max_ocr) * 100
  ) %>%
  filter(group %in% c("150.000", "250.000")) %>%
  mutate(group = forcats::fct_reorder(group, readr::parse_number(group)))

summary_df <-
  plot_df %>%
  summarize(
    median_max_ocr = median(max_ocr, na.rm = TRUE),
    q25 = quantile(max_ocr, 0.25, na.rm = TRUE),
    q75 = quantile(max_ocr, 0.75, na.rm = TRUE),
    .by = group
  )

ggplot(plot_df, aes(x = group, y = max_ocr)) +
  # median "bar" (thin column)
  geom_col(
    data = summary_df,
    aes(y = median_max_ocr, fill = group),
    width = 0.2,
    alpha = 0.4
  ) +
  # show all wells (jitter instead of weave)
  geom_jitter(
    aes(color = group),
    width = 0.08,
    height = 0,
    alpha = 0.8
  ) +
```

```
# median ± IQR interval (as a clean interval substitute)
geom_pointrange(
  data = summary_df,
  aes(y = median_max_ocr, ymin = q25, ymax = q75, color = group),
  linewidth = 0.6
) +
colorspace::scale_colour_discrete_divergingx(palette = "Geyser", rev = FALSE) +
colorspace::scale_fill_discrete_divergingx(palette = "Geyser", rev = FALSE) +
labs(
  subtitle = "Maximal OCR",
  x = "",
  y = "OCR (pmol/min)"
) +
theme_bw(base_size = 18)
```

Exercise 8

The plot above shows `maximal ocr`. Now make your own plot with 1) `basal_ocr` and 2) all groups except background. Make sure to order the group legend tidyly and have reable x-axis labels.

```
#use the code block above and change that code
```

Solution to Exercise 8

```
#solution 1 and 2 together
#lines with changes marked with #VB

param_set_maxcap_ocr <- c(init_ocr = "m3",
                          fccp_ocr = "m4",
                          amrot_ocr = "m9",
                          mon_ocr = "m12"
                         )

group_order <- seahtrue_output_donor_A %>% #VB
  pluck("rate_data",1) %>%
  pull(group) %>% unique()

plot_df <-
  seahtrue_output_donor_A %>%
  pluck("rate_data",1) %>%
  select(well, measurement, group, ocr = OCR_wave_bc) %>%
  pivot_wider(names_from = measurement,
              names_prefix = "m",
              values_from = ocr) %>%
  rename(all_of(param_set_maxcap_ocr)) %>%
  select(contains(c("well","group", "ocr"))) %>%
  #piped here to the mutate
  mutate(non_mito_ocr = amrot_ocr,
         basal_ocr = init_ocr - non_mito_ocr,
         max_ocr = fccp_ocr - amrot_ocr,
         spare_ocr = max_ocr - basal_ocr,
         spare_ocr_index = (spare_ocr / max_ocr)*100,
         basal_ocr_index = (basal_ocr/max_ocr)*100) %>%
  filter(group!= c("Background")) %>% #VB
  mutate(group = forcats::fct_reorder(group, readr::parse_number(group)))

summary_df <-
  plot_df %>%
  summarize(
    median_basal_ocr = median(basal_ocr), #VB
    q25 = quantile(basal_ocr, 0.25),
    q75 = quantile(basal_ocr, 0.75),
    .by = group
  )

ggplot(plot_df, aes(x = group, y = basal_ocr, color = group)) + #VB
  geom_bar(data = summary_df,
           mapping = aes(
             x = group,
             y = median_basal_ocr, #VB
             fill = group),
           stat="identity",
           alpha=0.4,
           width=0.4)+ #VB

  # replacement for ggdist::geom_weave()  #VB
```

9.5 Functional omics with Seahorse analysis - ATP interpretations

Mookerjee et al. (2017) J Biol Chem

```
library(tidyverse)

# raw path on github
url <- paste0(
  "https://raw.githubusercontent.com/vcjdeboer/seahtrue/develop-gerwin/inst/extdata/",
  "20200110_SciRep_PBMCs_donor_C.xlsx"
)

# Save it locally
download.file(
  url,
  destfile = "20200110_SciRep_PBMCs_donor_C.xlsx",
  mode = "wb")

# Load it into seahtrue
seahtrue_output_donor_C <-
  "20200110_SciRep_PBMCs_donor_C.xlsx" %>%
  seahtrue::revive_xfplate()
```

```
seahtrue_output_donor_C
```

```
df_space_C <-
  #first pluck the rate_data
  seahtrue_output_donor_C %>%
    pluck("rate_data", 1) %>%

    #only take the 200.000 cells/well group
    filter(group %in% c("200.000")) %>%

    #next call the calculate_space function
    seahtrue::calculate_space(

      # provide the injection info
      param_set_ocr = c(init_ocr = "m3",
                        fccp_ocr = "m4",
                        amrot_ocr = "m9",
                        mon_ocr = "m12"),
      param_set_ecar = c(init_ecar = "m3",
                         fccp_ecar = "m4",
                         amrot_ecar = "m9",
                         mon_ecar = "m12")
    ) %>%
    mutate(group = "donor_C")

df_space_A <-
  #first pluck the rate_data
  seahtrue_output_donor_A %>%
    pluck("rate_data", 1) %>%

    #only take the 200.000 cells/well group
    filter(group %in% c("200.000")) %>%

    #next call the calculate_space function
    seahtrue::calculate_space(

      # provide the injection info
      param_set_ocr = c(init_ocr = "m3",
                        fccp_ocr = "m4",
                        amrot_ocr = "m9",
                        mon_ocr = "m12"),
      param_set_ecar = c(init_ecar = "m3",
                         fccp_ecar = "m4",
                         amrot_ecar = "m9",
```

```
                          mon_ecar = "m12")
    ) %>%
    mutate(group = "donor_A")



#combine the two datasets in one df
df_space_total <- bind_rows(df_space_C, df_space_A)

print("df_space_total is generated")
```

Exercise x9

Verify that the correct measurements were put as arguments for the `calculate_space()` function

Solution to Exercise x9

Since we do not take the average of the three measurements for each phase, because we think that the cells reach a steady state after injection after a certain time interval, we take one measurement for each phase. In this experiment, we injected fccp, AM/rot and monension consecutively. The fccp injection typically give the highest value allready after one measurement so we take measurement 4 (m4). for the others we take the third measurement in the phase (m3, m9 and m12)
param_set_ocr = c(init_ocr = "m3", fccp_ocr = "m4", amrot_ocr = "m9", mon_ocr = "m12"),

```
my_palette<- c("#1f78b4","#33a02c")

df_space_total %>%
  seahtrue::plot_bioenergetic_trajectory(palette = my_palette)
```

Exercise x10

Now that we generated the trajectory through ATP space, explain what you observe. How does the trajectory change for each individual? What is the biggest difference between donor A and C

Solution to Exercise x10

please discuss with one of your lecturers

```
my_palette<- c("#1f78b4","#33a02c")

df_space_total %>%
  seahtrue::plot_bioenergetic_space(palette = my_palette)
```

Exercise x11

Explain the difference between the two spaces between donor A and B. Also, what does the dashed line represent?

Solution to Exercise x11

please discuss with one of your lecturers

Exercise ADVANCED x12

We have another dataset where we isolated T cells from an individual and stimulated the cells with CD3/CD28 (= IC). Also, we injected a number of chemical compounds. Please focus for now on the "IC + 1.5 uM BAM15" and "1.5 uM BAM15" group. Instead of the uncoupler FCCP we now used BAM15 instead which is more gentle on the cell membrane.
Keep the `fccp_ocr` and `fccp_ecar` nomenclature for your param_sets. The function does not recognize `bam15_ocr` or `bam15_ecar`.
Find out which measurements you should use to calculate the space.
Filter out well C10. This is a technical outlier.
Use "agilent" as the atp conversion model.
Plot the ATP space for the activated and non-activated T cells.

- What do you observe?
- What does it mean when the basal J_ATP (the dot in the space) is more closer to the dashed line?

```
#load the data
# raw path on github
url <- paste0(
  "https://raw.githubusercontent.com/vcjdeboer/seahtrue/develop-gerwin/inst/extdata/",
  "20251102_Tcell_Activation_VP.xlsx"
)

# Save it locally
download.file(
  url,
  destfile = "20251102_Tcell_Activation_VP.xlsx",
  mode = "wb")

# Load it into seahtrue
df_t_cell_activation <-
  "20251102_Tcell_Activation_VP.xlsx" %>%
  seahtrue::revive_xfplate()
```

Solution to Exercise ADVANCED x12

We want to see the difference between activate T cells and not activated, so we should take the measurement 9 where the cells were activated or not (at measurement three both groups are still the same!).

```
#use this param sets
my_param_set_ocr =  c(init_ocr = "m9", om_ocr = "m12",
                      fccp_ocr = "m15", amrot_ocr = "m18")
my_param_set_ecar = c(init_ecar = "m9", om_ecar = "m12",
                      fccp_ecar = "m15", amrot_ecar = "m18")

my_rate <- df_t_cell_activation %>%
              purrr::pluck("rate_data", 1) %>%
              dplyr::filter(well != "C10") %>%
              dplyr::filter(stringr::str_detect(group, "1.5 uM BAM15"))

df_space_tcell <- calculate_space(rate = my_rate,
                                  param_set_ocr = my_param_set_ocr,
                                  param_set_ecar = my_param_set_ecar,
                                  conversion_model = "agilent"
                                  )


my_palette <- c("#e41a1c", "#984ea3")

df_space_tcell %>%
  seahtrue::plot_bioenergetic_space(palette = my_palette)
```

10 Multiple experiments

10.1 Gimme more plates

```
library(tidyverse)

# Raw path on GitHub
url <- paste0(
  "https://raw.githubusercontent.com/vcjdeboer/seahtrue/develop-gerwin/inst/extdata/",
  "xf_3.rda"
)

# Save it locally
download.file(
  url,
  destfile = "xf_3.rda",
  mode = "wb"
)

# Load it into the R environment
load("xf_3.rda")

xf_3
```

```
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data))
```

```
webr::install("ggridges")

xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group != "Background") %>%
  ggplot(aes(x = O2_mmHg, y = plate_id))+
    ggridges::geom_density_ridges()+
    facet_wrap(~forcats::fct_reorder(group,
                                     parse_number(group)))
```

-
-
-

```
library(ggridges)

xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
```

```
  filter(group == "50.000") %>%
  ggplot(aes(x = O2_mmHg, y = plate_id,
  fill = ifelse(after_stat( x > 154),
            "above 154", "below 154")))+
  stat_density_ridges(geom = "density_ridges_gradient",
      quantile_lines = TRUE,
      quantiles = 2) +
  theme_ridges() +
  scale_fill_manual(values = c("red", "gray70"),
                    name = NULL)
```

```
library(RColorBrewer)
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group == "50.000") %>%
  filter(str_detect(plate_id,"V01947"))%>%
  ggplot(aes(x = minutes, y = O2_mmHg,
  color = well))+
    geom_point()+
    scale_color_manual(values =
        colorRampPalette(
          brewer.pal(4, "PuOr"))(14))+
    theme_bw(base_size= 18)+
    labs(y = "O2 (mmHg)",
         x = "time (minutes)")
```

```
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group == "50.000") %>%
  filter(str_detect(plate_id,"V01947"))%>%
```

```
ggplot(aes(x = minutes, y = O2_mmHg,
color = well))+
  geom_point()+
  geom_text(data = . %>%
              filter(minutes == max(minutes)) %>%
              filter(O2_mmHg >153),
              aes(label = well),
              vjust = 2.4,
              show.legend = FALSE)+
  scale_color_manual(values =
      colorRampPalette(
        brewer.pal(4, "PuOr"))(14))+
  theme_bw(base_size= 18)+
  labs(y = "O2 (mmHg)",
       x = "time (minutes)")
```

Exercise 1

Make the same scatter plot for the other two plates for the `50.000` group, and observe if there are wells with abnormally high O2.

```
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group == "50.000") #%>%
```

Solution to Exercise 1

```
#solution for experiment 2
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group == "50.000") %>%
  filter(str_detect(plate_id,"V01941")) %>% #change 7 to 1
  ggplot(aes(x = minutes, y = O2_mmHg,
  color = well))+
    geom_point()+
    geom_text(data = . %>%
                filter(minutes == max(minutes)) %>%
                filter(O2_mmHg >153),
                aes(label = well),
                vjust = 2.4)+
    scale_color_manual(values =
        colorRampPalette(
          brewer.pal(4, "PuOr"))(14))+
    theme_bw(base_size= 18)+
    labs(y = "O2 (mmHg)",
         x = "time (minutes)")

# wells C02 and F01 are high
```

```
#solution for experiment 3
xf_3 %>%
  select(plate_id, raw_data) %>%
  unnest(c(raw_data)) %>%
  filter(group == "50.000") %>%
  filter(str_detect(plate_id,"V01744")) %>% #change to 1744
  ggplot(aes(x = minutes, y = O2_mmHg,
  color = well))+
    geom_point()+
    scale_color_manual(values =
        colorRampPalette(
          brewer.pal(4, "PuOr"))(14))+
    theme_bw(base_size= 18)+
    labs(y = "O2 (mmHg)",
         x = "time (minutes)")

#O2 mmHg is similar in all wells (no high O2 outlier)
```

11 Download files and analyze

```
print("webr is working!")
```

11.1 Files Available on GitHub

extdata folder

```
#  USER: Set the exact file name here (must match GitHub!)
filename <- "20200110_SciRep_PBMCs_donor_C.xlsx"

#  GitHub folder (keep unchanged unless moving files)
github_folder <- paste0(
  "https://raw.githubusercontent.com/vcjdeboer/seahtrue/develop-gerwin/inst/extdata/"
)

#  Construct full download URL
url <- paste0(github_folder, filename)

#  Download file into webr virtual filesystem
download.file(
  url,
  destfile = filename,
  mode = "wb"
)

#  Load the file using seahtrue
seahtrue_output <- filename |>
  seahtrue::revive_xfplate()

#  Inspect structure
seahtrue_output |>
  dplyr::glimpse()
```

11.2 Plot the plate layout

```
seahtrue_output %>%
  pluck("raw_data", 1) %>%
  seahtrue::sketch_plate(reorder_legend = TRUE)
```

11.3 Plot the rate data

```
seahtrue_output %>%
  purrr::pluck("rate_data", 1) |>
  sketch_rate(
         param = "OCR",
         normalize = FALSE,
         take_group_mean = TRUE,
         reorder_legend = TRUE
     )
```

11.4 Plot the raw data histogram

```
seahtrue_output %>%
  sketch_assimilate_raw(param = "O2_mmHg")

seahtrue_output %>%
  sketch_assimilate_raw(param = "pH")
```

11.5 Plot the measurement rate data

```
seahtrue_output %>%
         sketch_assimilate_rate(
             param = "OCR",
             my_measurements = c(3, 4, 9, 12)
       )
```

11.6 Plot the space plot

```
# first check out the exact group names

seahtrue_output |>
  pluck("rate_data", 1) |>
  pull(group) |> unique()
```

```
# default parameter assignement for a mito stress test

param_set_ocr <- c(
  init_ocr = "m3",
  om_ocr = "m6",
  fccp_ocr = "m7",
  amrot_ocr = "m12")

param_set_ecar <- c(
  init_ocr = "m3",
  om_ocr = "m6",
  fccp_ocr = "m7",
  amrot_ocr = "m12")
```

```


#  STEP 1: Set the groups to include (max 4 recommended)
selected_groups <- c("200.000", "300.000")

#  STEP 2: Provide injection parameters (keep naming consistent)
param_set_ocr <- c(
  init_ocr = "m3",
  fccp_ocr = "m4",
  amrot_ocr = "m9",
  mon_ocr = "m12"
)

param_set_ecar <- c(
  init_ecar = "m3",
  fccp_ecar = "m4",
  amrot_ecar = "m9",
  mon_ecar = "m12"
)
```

```
#  STEP 3: Create the space plot input data
df_space_user <- seahtrue_output |>
  pluck("rate_data", 1) |>
  filter(group %in% selected_groups) |>
  seahtrue::calculate_space(
    param_set_ocr = param_set_ocr,
    param_set_ecar = param_set_ecar
  )
#  STEP 4: Plot the space plot
seahtrue::plot_bioenergetic_space(df_space_user)
```